



**Cofinanțat de
Uniunea Europeană**



Program: „Programul Educație și Ocupare“

Prioritate: P07 „Creșterea calității ofertei de educație și formare profesională pentru asigurarea echității sistemului și o mai bună adaptare la dinamica pieței muncii și la provocările inovării și progresului tehnologic“

Obiectiv specific: ESO4.5_Îmbunătățirea calității, a caracterului incluziv, a eficacității și a relevanței sistemelor de educație și formare pentru piața muncii, inclusiv prin validarea învățării nonformale și informale, pentru a sprijini dobândirea de competențe-cheie, inclusiv de competențe de antreprenoriat și digitale, precum și prin promovarea introducerii sistemelor de formare duală și a sistemelor de ucenicie

Apel de proiecte: „: Promovarea dezvoltării programelor de studii terțiare de înaltă calitate, flexibile și corelate cu cerințele pieței muncii - STAGII STUDENȚI- Regiuni mai puțin dezvoltate“

Cod SMIS: 302355

UNIVERSITATEA POLITEHNICA TIMIȘOARA

FACULTATEA DE MECANICĂ

Departamentul Mecatronică și Robotică

Specializarea Mecatronică și Robotică

DOSAR DE PRACTICĂ

Nume și prenume student

Tăian Adrian-Ciprian

An de studiu / Specializare

Anul II / Mecatronică și Robotică

An universitar 2025-2026

Informații generale despre desfășurarea practicii

Nume partener de practică	
Adresa partenerului de practică	
Departamentul / secția în care se derulează stagiul de practică	
Adresa unde are loc stagiul de practică	
Nume și prenume tutore / funcția (îndrumător de practică)	
Nume și prenume cadru didactic supervizor (responsabil de practică)	
Perioada desfășurării stagiului de practică	de la: 20.07.2026 (zi/lună/an)
	până la: 28.08.2026 (zi/lună/an)
	ore/zi: 8h/zi

Jurnal al activității desfășurate în fiecare zi de practică

Acest tabel se completează **zilnic/centralizat pe săptămâni** de către student.

Data	Nr. ore	Descrierea pe scurt a activității realizate
20.07.2026	8	Instructaj de securitate și sănătate în muncă. Prezentarea partenerului de practică și a temei de lucru: detecția automată a schimbărilor între două ortofotoplanuri de dronă ale aceleiași zone, ridicate la momente diferite. Stabilirea denumirii proiectului (Argus Custode) și a limitelor lui: se lucrează pe date publice și sintetice până la primirea unor zboruri reale.
21.07.2026	8	Documentare privind fotogrammetria de dronă: ortofotoplan, distanța de eșantionare la sol (GSD), suprapunerea frontală și laterală dintre cadre. Studiul formatului GeoTIFF și al sistemelor de coordonate folosite în România (WGS84 / EPSG:4326 și Stereografic 1970).
22.07.2026	8	Studiul metodelor de detecție a schimbărilor: diferență directă de pixeli, clasificare supervizată și detecție de anomalii. Am ales detecția de anomalii cu Isolation Forest, întrucât nu există și nu se pot obține etichete de antrenare pentru zonele modificate.
23.07.2026	8	Redactarea planului de implementare pe șapte faze și stabilirea ordinii de execuție 4–6–5–3–1–2–7, adică invers față de fluxul datelor: se începe cu ceea ce nu depinde de hardware și de date de zbor inexistente. Deschiderea registrului de decizii și a listei de întrebări deschise, fiecare cu soluție de rezervă.
24.07.2026	8	Inițializarea depozitului Git și a structurii de directoare (backend, frontend, teste, date). Mediu Python izolat, cu versiuni fixate ale bibliotecilor (numpy, rasterio, scikit-learn, shapely, FastAPI, pytest). Verificare: importurile trec fără eroare, iar fișierul de dependențe conține versiuni exacte, nu intervale.
27.07.2026	8	Descărcarea unui ortofotoplan public de dronă, cu licență permisivă, folosit ca imagine de referință. Verificarea sistemului de coordonate și a dimensiunilor. Configurarea excluderii rasterelor din versionare, pentru ca depozitul să rămână ușor de clonat pe altă mașină.
28.07.2026	8	Construirea perechii sintetice de test: o copie a imaginii de referință cu patru schimbări injectate manual (structură demolată, obiect nou apărut, vegetație îndepărtată, șanț de excavație) și un fișier cu poligoanele lor exacte. Regula stabilită atunci și respectată până la final: adevărul cunoscut nu intră niciodată în antrenare, doar în evaluare.
29.07.2026	8	Implementarea extragerii de trăsături: împărțirea rasterului în petice de 32×32 pixeli și calcularea a douăsprezece mărimi pe fiecare petic (culoare medie, varianță locală și gradient spațial, pe fiecare dintre cele trei canale). Totul vectorizat cu numpy, fără buclă Python peste petice, pentru a rămâne sub zece secunde pe o imagine întreagă.
30.07.2026	8	Implementarea detecției propriu-zise: antrenarea unui Isolation Forest pe peticele imaginii de referință, scorarea peticelor imaginii ulterioare și transformarea scorurilor în poligoane georeferențiate.

Data	Nr. ore	Descrierea pe scurt a activității realizate
		Primul rezultat măsurat: niciuna dintre cele patru schimbări reale nu apare în primii douăzeci de candidați.
31.07.2026	8	Diagnosticarea rezultatului slab: trăsăturile absolute descriu cât de neobișnuit arată un petic în sine, nu cât de mult s-a schimbat. Trecerea la trăsături de diferență, adică valoarea absolută a diferenței dintre trăsăturile celor două imagini. Rezultat imediat: trei din patru schimbări în primii douăzeci de candidați și patru din patru în primii cincizeci.
03.08.2026	8	Studiul formatului COG (Cloud Optimized GeoTIFF) și al protocolului de dale XYZ folosit de bibliotecile de hartă. Decizie: dalele se generează dinamic din raster, cu păstrarea în memorie a fișierului deschis, în locul unei piramide de imagini pregenerate care ar fi ocupat spațiu și ar fi trebuit reconstruită la fiecare procesare.
04.08.2026	8	Implementarea servirii de dale: citirea unei ferestre din raster cu rasterio, redimensionarea la 256×256 pixeli și returnarea unei imagini PNG. Măsurarea latenței pe dală și adăugarea unui cache pentru fișierele deschise, ca să nu se redeschidă rasterul la fiecare cerere.
05.08.2026	8	Realizarea serviciului web cu FastAPI: bază de date SQLite pentru metadate, cu jurnalizare în regim WAL, și executarea procesării în fundal, astfel încât o cerere lungă să nu blocheze celelalte cereri. Decizie consemnată explicit: nu se introduce o coadă de mesaje externă pentru o singură mașină de lucru.
06.08.2026	8	Definirea rutelor interfeței de programare: creare de zbor, pornirea procesării, interogarea stării, obținerea rezultatului și servirea dalelor. Adăugarea unei bucle de migrare a schemei la pornire, astfel încât structura bazei de date să poată evolua fără pierderea datelor existente.
07.08.2026	8	Primele teste automate pentru extragerea de trăsături și pentru detecție. Am descoperit că un test al algoritmului central trecea deși compara imaginea cu ea însăși, deci nu demonstra nimic. Testul a fost rescris cu o schimbare reală injectată și cu o verificare a rangului acesteia.

Data	Nr. ore	Descrierea pe scurt a activității realizate
10.08.2026	8	Inițializarea interfeței cu React și Vite și integrarea bibliotecii de hartă MapLibre GL. Afișarea ortofotoplanului folosind dalele servite de propriul serviciu web, nu un furnizor extern de hărți.
11.08.2026	8	Suprapunerea peste raster a stratului cu poligoanele de anomalie, colorate în funcție de scor, și adăugarea unui panou lateral cu lista candidaților, ordonată după rang. Selectarea unui candidat din listă centrează harta pe zona respectivă.
12.08.2026	8	Implementarea comparației înainte / după printr-un cursor care modifică transparența unui strat deja încărcat, nu sursa de date. Motivul: schimbarea sursei ar declanșa cereri de rețea la fiecare pas al cursorului, iar comparația ar deveni sacadată.

Data	Nr. ore	Descrierea pe scurt a activității realizate
13.08.2026	8	Reorganizarea interfeței pe trei secțiuni de lucru (Anomalii, Comparatie, Ingestie) și primul pas serios de accesibilitate: navigare completă de la tastatură și reducerea barei de secțiuni de la cincisprezece opriri de Tab la una singură, prin tehnica tabindex-ului mobil.
14.08.2026	8	Dialogul cu lista completă de anomalii, realizat cu elementul nativ de dialog, cu mutarea focusului pe titlu la deschidere, capcană de focus și dezactivarea conținutului din spate. Verificare manuală: treizeci de apăsări de Tab nu scot focusul din dialog.
17.08.2026	8	Prima rulare completă a lanțului de detecție și consemnarea cifrei obținute în jurnalul de proiect. Incident de proces: agentul de execuție delegat a comis și a trimis pe ramura principală aproximativ cinci sute de linii nerevizuite. Regulă adăugată în fișierul de instrucțiuni al proiectului: operațiile Git rămân la om, agentul doar editează fișiere.
18.08.2026	8	Revizuirea, commit cu commit, a modificărilor trimise fără control și curățarea consecințelor. Rescrierea listei de sarcini într-o formă executabilă: fiecare sarcină are obiectiv, comandă de verificare, criteriu de terminare și procedură de revenire.
19.08.2026	8	Verificare independentă a rezultatului pe o altă mașină, cu parametrii implicați din cod: recall-ul real este de două din patru în primii douăzeci de candidați și de trei din patru în primii cincizeci, nu trei din patru cum fusese notat inițial. Cifra a fost corectată în jurnal, nu ascunsă.
20.08.2026	8	Pregătirea punerii în funcțiune: fișier de containerizare pentru serviciul web, configurarea găzduirii pentru interfață și separarea adresei serviciului într-o variabilă de mediu, astfel încât aceeași bază de cod să funcționeze local și în producție.
21.08.2026	8	Punerea în funcțiune publică a aplicației. Trei eșecuri reale rezolvate pe parcurs: lipsa unui pachet numpy precompilat pentru versiunea de Python aleasă și două runde de memorie epuizată pe planul gratuit de 512 MB, rezolvate prin citirea rasterului pe ferestre și prin reducerea la 3000 de pixeli pe latura lungă. Efect secundar neplanificat: recall-ul a urcat la patru din patru.
24.08.2026	8	Verificarea din exterior a celor două adrese publice, fără autentificare. Dezactivarea protecției implicite care bloca accesul public la interfață. Adăugarea unui fișier de attribute Git și renormalizarea depozitului, după ce s-a constatat că terminatorii de linie difereau de la o mașină la alta.
25.08.2026	8	Faza de validare a fotografiilor la încărcare: citirea metadatelor EXIF (poziție GPS, altitudine, distanță focală), calcularea unui scor de claritate prin varianța laplacianului și estimarea suprapunerii dintre cadre consecutive, cu verdict pe întregul set. Verificarea cursorului de comparație cu un test automat de browser: patruzeci și unu de evenimente de mișcare, zero cereri de rețea noi.
26.08.2026	8	Panoul de încărcare în interfață, operabil integral de la tastatură, cu echivalent de tastatură pentru zona de tragere a fișierelor și cu

Data	Nr. ore	Descrierea pe scurt a activității realizate
		anunțarea verdictului printr-o regiune activă, nu doar prin culoare. Redactarea listei de întrebări pentru coordonatorul de practică: definiția schimbării, tipul dronei, hardware-ul disponibil, confidențialitatea datelor.
27.08.2026	8	Împachetarea aplicației ca program Windows de sine stătător: executabil generat cu PyInstaller și fereastră WebView2, serviciul web servind și interfața din aceeași origine. Defect reparat: într-o aplicație fără consolă, fluxul standard de ieșire este inexistent, iar orice afișare în consolă oprea pornirea programului.
28.08.2026	8	Suita de teste de interfață în integrare continuă: 48 de teste de browser care acoperă navigarea de la tastatură, gestionarea focusului, dialogul, reșezarea la 320 de pixeli, echivalentul textual al hărții și o scanare automată de accesibilitate, plus un test de contract între serviciul simulat și cel real. Bilanț final și redactarea scenariului de verificare cu cititor de ecran, rămas ca pas următor.

Proiect de practică

Argus Custode — sistem de detecție automată a schimbărilor între ortofotoplanuri de dronă

Proiect de practică — Universitatea Politehnica Timișoara, Facultatea de Mecanică

1. Introducere

1.1. Contextul temei

Monitorizarea periodică a unui amplasament — un șantier, o carieră, un depozit de materiale, un tronson de infrastructură — se face astăzi tot mai des cu aeronave fără pilot la bord. Un zbor fotogrammetric acoperă câteva zeci de hectare în mai puțin de o oră și produce, după prelucrare, un ortofotoplan: o imagine aeriană corectată geometric, în care fiecare pixel are coordonate cunoscute și distanțele măsurate pe imagine corespund distanțelor de pe teren.

Problema apare la pasul următor. Când aceeași zonă este survolată de două ori, la distanță de câteva săptămâni, rezultă două imagini de câteva mii de pixeli pe latură, aproape identice. Diferențele care contează cu adevărat — o construcție nouă, o structură demolată, un depozit de material care s-a mutat, un șanț săpat între timp — ocupă împreună o fracțiune de procent din suprafață. Compararea lor se face în practică vizual, de către un operator care parcurge imaginea porțiune cu porțiune. Este o activitate lentă, obositoare și dependentă de atenția unui om la sfârșit de program.

1.2. Obiectivul lucrării

Obiectivul stagiului a fost realizarea unei aplicații care primește două ortofotoplanuri ale aceleiași zone, ridicate la momente diferite, și returnează o listă ordonată de zone candidate la schimbare, fiecare cu poligonul ei geografic și cu un scor de anormalitate. Aplicația nu înlocuiește operatorul uman, ci îi schimbă sarcina: în loc să caute schimbările pe toată suprafața, verifică o listă scurtă, ordonată după cât de probabil este ca fiecare intrare să fie o schimbare reală.

Din această formulare decurg trei cerințe secundare, care au ghidat toate deciziile tehnice ulterioare: rezultatul trebuie să fie măsurabil, adică să existe o cifră care spune cât de bine funcționează detecția; aplicația trebuie să fie folosibilă fără instalare și fără cunoștințe de programare; iar codul trebuie să poată fi reluat pe altă mașină, de altcineva, pornind de la documentația proprie.

1.3. Rezultatul obținut

La finalul stagiului aplicația este funcțională și livrată în două forme: o variantă web, accesibilă printr-un link public, fără cont și fără instalare, și un program Windows de sine stătător, care rulează local și nu are nevoie de conexiune la internet. Pe perechea de imagini de test cu adevăr cunoscut, toate cele patru schimbări injectate sunt regăsite în primii zece candidați propuși, iar cifra este remăsurată automat la fiecare modificare a codului.

2. Fundamentare teoretică

2.1. Ortofotoplanul

Fotografia aeriană brută nu este o hartă. Obiectivul introduce distorsiuni, aeronava nu este perfect orizontală în momentul declanșării, iar relieful face ca punctele mai înalte să apară deplasate față de poziția lor reală. Ortorectificarea corectează aceste efecte folosind un model digital al terenului obținut din suprapunerea cadrelor, iar rezultatul — ortofotoplanul — poate fi măsurat direct.

Doi parametri descriu calitatea unui asemenea produs. Distanța de eșantionare la sol (GSD) arată câți centimetri de teren revin unui pixel și depinde de înălțimea de zbor, de distanța focală și de dimensiunea senzorului. Suprapunerea dintre cadre consecutive determină dacă reconstrucția este posibilă: sub aproximativ 60% suprapunere frontală, algoritmi de potrivire nu găsesc destule puncte comune, iar zborul devine inutilizabil. Ambele mărimi revin în capitolul 6, la validarea datelor de intrare.

2.2. Abordări posibile pentru detecția schimbărilor

Literatura de specialitate propune trei familii de metode, cu cerințe de date foarte diferite.

- Diferența directă de pixeli scade o imagine din cealaltă și marchează zonele unde diferența depășește un prag. Este cea mai simplă metodă și, pe date reale, cea mai înșelătoare: o umbră deplasată cu două ore de soare produce o diferență mai mare decât o clădire nouă.
- Clasificarea supervizată antrenează un model pe exemple etichetate de „schimbat” și „neschimbat”. Dă cele mai bune rezultate atunci când datele etichetate există. În cazul de față nu existau și nu puteau fi obținute: nu aveam zboruri reale, iar etichetarea manuală a câtorva mii de petice de imagine ar fi consumat tot stagiul.
- Detecția de anomalii nu are nevoie de etichete. Modelul învață cum arată normalul din datele însele și marchează ce se abate de la el. Este metoda potrivită atunci când schimbările sunt, prin definiție, rare — exact situația de aici.

2.3. Isolation Forest

Metoda aleasă, Isolation Forest, pornește de la o observație simplă: o valoare atipică este mai ușor de izolat de restul datelor decât una obișnuită. Algoritmul construiește un număr mare de arbori de partiționare aleatoare; la fiecare pas alege o trăsătură la întâmplare și un prag la întâmplare între minimul și maximul ei, până când fiecare observație rămâne singură într-o frunză. Punctele izolate în puține tăieturi sunt cele care se află departe de aglomerarea principală.

Scorul de anomalie al unei observații este calculat din adâncimea medie la care aceasta este izolată, mediată peste toți arborii. Avantajele care au contat pentru acest proiect sunt trei: nu cere etichete, are cost liniar în numărul de observații, și nu presupune nimic despre forma distribuției datelor. Dezavantajul, la fel de real, este că modelul nu poate explica de ce a considerat ceva anormal — motiv pentru care rezultatul este prezentat ca listă de candidați de verificat, nu ca verdict.

3. Analiza cerințelor și planificarea lucrării

3.1. Necunoscutele de la început

La începutul stagiului lipseau exact lucrurile de care depinde în mod normal un asemenea proiect: nu exista acces la o dronă, nu existau zboruri arhivate, nu era stabilit ce hardware va fi disponibil pentru prelucrare și nu era definit ce înseamnă „schimbare” pentru beneficiar — construcții noi, creșterea vegetației, deplasarea stocurilor de material sau eroziune.

Fiecare necunoscută a fost consemnată într-o listă de întrebări deschise, împreună cu faza pe care o blochează și cu soluția de rezervă aplicabilă dacă răspunsul întârzie. Regula stabilită atunci a fost că nicio întrebare deschisă nu are voie să oprească lucrul: dacă oprește, primește o soluție de rezervă și se merge mai departe. Această regulă a permis ca aplicația să ajungă funcțională și publică pe date sintetice înainte de a primi vreun răspuns.

3.2. Fazele proiectului și ordinea de execuție

Lucrarea a fost împărțită în șapte faze, urmând fluxul natural al datelor: ingestia și validarea fotografiilor (1), fotogrammetria (2), depozitarea rasterelor (3), detecția schimbărilor (4), interfața de programare (5), harta interactivă (6) și punerea în funcțiune (7).

Ordinea de execuție a fost însă deliberat inversată: 4 – 6 – 5 – 3 – 1 – 2 – 7. Motivul este că fazele 1 și 2 depind de hardware necunoscut și de date de zbor inexistente, în timp ce faza 4, care conține partea originală a lucrării, se putea porni imediat pe o pereche de imagini

construită în laborator. Începerea cu faza care poartă cel mai mare risc tehnic a însemnat că, dacă metoda de detecție nu ar fi funcționat, aş fi aflat în prima săptămână, nu în ultima.

3.3. Registrul de decizii

Fiecare alegere tehnică netrivială a fost consemnată într-un registru de decizii, cu data, motivul şi consecinţa asumată. Registrul a ajuns la optsprezece intrări. Rolul lui nu este ceremonial: de două ori pe parcursul stagiului a fost nevoie să revin asupra unei alegeri, iar prezenţa motivului scris a arătat imediat dacă premisa iniţială mai era valabilă sau nu. Cele mai importante decizii sunt rezumate în tabelul următor.

Decizie	Motiv	Consecinţă asumată
Detecţie nesupervizată, nu clasificare	Nu există şi nu se pot obţine etichete	Rezultatul este o listă de candidaţi, nu un verdict
Trăsături de diferenţă, nu absolute	Absolutele descriu textura, nu schimbarea	Este obligatoriu ca imaginile să fie aliniate
Dale generate dinamic din raster	Piramida pregenerată ocupă spaţiu şi se învecheşte	Cost de calcul la fiecare cerere de dală
Prelucrare în fundal, fără coadă externă	O singură maşină, un singur utilizator	Nu scalează la execuţii simultane multiple
SQLite în regim WAL, nu bază de date server	Metadate puţine, instalare zero	Fără scriitori concurenţi din procese diferite
Praguri ca parametri, nu constante în cod	Calibrarea reală vine odată cu datele reale	Mai multe argumente de configurat

Tabelul 1. Extras din registrul de decizii al proiectului

4. Arhitectura aplicaţiei

4.1. Vedere de ansamblu

Aplicaţia este alcătuită din trei componente: un serviciu web care conţine prelucrarea şi expune o interfaţă de programare, o interfaţă de utilizator care rulează în browser şi un ambalaj de desktop care le porneşte pe amândouă local. Aceeaşi bază de cod produce atât varianta web, cât şi programul Windows; diferă doar modul de construire.

Fluxul unei prelucrări este următorul: utilizatorul creează un zbor, încarcă cele două rastere, porneşte procesarea, iar aceasta se execută în fundal, astfel încât cererea utilizatorului să revină imediat. Starea prelucrării se interoghează periodic, iar la final rezultatul — un fişier GeoJSON cu poligoanele candidate şi scorurile lor — este disponibil pentru afişare pe hartă.

4.2. Serviciul web

Serviciul este construit cu FastAPI şi expune treisprezece rute HTTP, grupate în trei familii: administrarea zborurilor, încărcarea şi validarea fotografiilor, şi servirea dalelor de imagine. Metadatele se păstrează într-o bază SQLite pusă în regim de jurnalizare WAL, care permite citirea în timp ce o prelucrare scrie. Schema bazei de date se actualizează la schimbarea versiunii de cod, iar pentru fiecare versiune se creează o copie de rezervă. Pentru mai multe detalii, consultaţi documentaţia proiectului.

bucă de migrare, ceea ce a permis adăugarea de coloane noi pe parcursul stagiului fără pierderea datelor deja existente.

Alegerea de a executa prelucrarea în fundal, cu mecanismul propriu al cadrului de lucru, în locul unei cozi de mesaje externe, a fost luată conștient. O coadă de mesaje ar fi adăugat două servicii suplimentare de administrat, pentru un caz de utilizare cu un singur utilizator și o singură mașină. Limita acestei alegeri este consemnată în registru: soluția nu suportă mai multe prelucrări grele simultane.

4.3. Servirea imaginilor

Un ortofotoplan are câteva mii de pixeli pe latură și nu poate fi trimis integral către browser. Soluția standard este împărțirea în dale de 256×256 pixeli, organizate pe niveluri de mărire. În loc să pregenerez piramida de dale ca fișiere pe disc, serviciul o produce la cerere: pentru fiecare dală se calculează fereastra corespunzătoare din raster, se citește doar acea fereastră, se redimensionează și se returnează ca imagine PNG. Fișierele deschise sunt păstrate într-un cache, ca rasterul să nu fie redeschis la fiecare cerere.

Avantajul este că nu există un pas separat de pregătire care să trebuiască repetat după fiecare prelucrare și nu se ocupă spațiu suplimentar. Costul este calculul făcut la fiecare cerere, acceptabil pentru numărul de utilizatori vizat.

4.4. Interfața de utilizator

Interfața este o aplicație React construită cu Vite, care afișează harta prin MapLibre GL. Ortofotoplanul este un strat raster alimentat din dalele proprii, iar rezultatul detecției un strat vectorial suprapus, colorat în funcție de scor. Interfața este organizată pe trei secțiuni de lucru: Anomalii, cu lista candidaților; Comparatie, cu cursorul înainte / după; și Ingestie, cu încărcarea și validarea fotografiilor.

Cursorul de comparație modifică transparența unui strat deja încărcat, nu sursa lui de date. Distincția pare minoră, dar decide dacă instrumentul este utilizabil: schimbarea sursei ar declanșa cereri de rețea la fiecare pas al cursorului, iar comparația ar deveni sacadată. Comportamentul este verificat automat — un test de browser execută o tragere realistă a cursorului și numără cererile de rețea apărute în acest interval.

4.5. Programul de desktop

Pentru situațiile în care datele nu pot părăsi rețeaua beneficiarului, aplicația este împachetată și ca program Windows de sine stătător. Executabilul este generat cu PyInstaller, iar interfața este afișată într-o fereastră WebView2, componentă preinstalată pe Windows 11. Serviciul web pornește pe un port local ales la rulare și servește și interfața din aceeași origine, ceea ce

elimină complet problemele de partajare între origini diferite. Datele și fișierul de jurnal se scriu în directorul de aplicație al utilizatorului.

5. Metoda de detecție a schimbărilor

5.1. Împărțirea în petice și extragerea trăsăturilor

Rasterul este împărțit într-o grilă de petice de 32×32 pixeli. Dimensiunea este un compromis: petice mai mici cresc rezoluția detecției, dar și zgomotul, pentru că statistica pe puțini pixeli devine instabilă; petice mai mari netezesc zgomotul, dar înghit schimbările mici. Dimensiunea este parametru, nu constantă scrisă în cod.

Pentru fiecare petic se calculează douăsprezece mărimi: culoarea medie, varianța locală și amplitudinea gradientului spațial, fiecare pe cele trei canale de culoare. Culoarea medie surprinde schimbările de material, varianța surprinde schimbările de textură, iar gradientul surprinde apariția sau dispariția muchiilor — un acoperiș nou, o margine de excavație. Calculul este vectorizat integral cu numpy, prin reorganizarea matricei în blocuri și reduceri pe axe, fără nicio buclă Python peste petice; pe o imagine întreagă se execută în câteva secunde.

5.2. Rezultatul negativ care a schimbat metoda

Prima variantă implementată antrena modelul pe trăsăturile peticelor din imaginea de referință și scora, cu același model, trăsăturile peticelor din imaginea ulterioară. Rezultatul a fost fără echivoc: niciuna dintre cele patru schimbări injectate nu apărea în primii douăzeci de candidați, iar toate patru apăreau abia în primii trei mii.

Cauza, odată identificată, este evidentă în retrospectivă. Un model antrenat pe trăsături absolute răspunde la întrebarea „cât de neobișnuit arată acest petic în raport cu restul imaginii?”. Un acoperiș care se demolează și lasă în loc pământ bătătorit nu devine neobișnuit — pământul bătătorit este ceva foarte comun într-o imagine aeriană. Modelul învățase să găsească texturi rare, nu schimbări.

Corecția a fost trecerea la trăsături de diferență: pentru fiecare petic se calculează valoarea absolută a diferenței dintre vectorul de trăsături al imaginii ulterioare și cel al imaginii de referință, iar modelul se antrenează și scorează pe aceste diferențe. Întrebarea la care răspunde modelul devine astfel „cât de mult s-a schimbat acest petic în raport cu cât se schimbă restul imaginii?”, adică exact întrebarea proiectului. Efectul măsurat este prezentat în tabelul 2.

Variantă de trăsături	Regăsite în primii 20	Regăsite în primii 50
Trăsături absolute, model antrenat pe imaginea de referință	0 din 4	0 din 4 (4 din 4 abia la 3000)
Trăsături de diferență între cele două imagini	3 din 4	4 din 4

Tabelul 2. Efectul trecerii la trăsături de diferență, pe aceeași pereche de imagini

Este singura schimbare de metodă din tot proiectul și, în același timp, singura care a contat cu adevărat pentru rezultat. Restul lucrării — servirea imaginilor, interfața, împachetarea — reprezintă inginerie necesară în jurul acestei observații.

5.3. Măsurarea calității

O afirmație de tipul „aplicația detectează schimbări” nu înseamnă nimic dacă nu poate fi verificată. Pentru a produce o cifră, am construit o pereche de imagini cu adevăr cunoscut: o copie a ortofotoplanului de referință în care am introdus manual patru modificări distincte — o structură demolată, un obiect nou apărut, o zonă de vegetație îndepărtată și un șanț de excavație — și un fișier separat care conține poligoanele exacte ale acestora.

Regula respectată pe tot parcursul a fost că fișierul cu adevărul cunoscut nu intră niciodată în antrenare, ci numai în evaluare. Dacă adevărul se scurge în antrenare, cifra măsurată nu mai înseamnă nimic. Măsura folosită este proporția schimbărilor reale regăsite printre primii N candidați propuși de model — mărimea direct relevantă pentru modul în care este folosită aplicația, unde operatorul parcurge o listă scurtă.

Evaluarea nu se face manual, ci este un test automat executat la fiecare modificare a codului. Testul verifică inclusiv că evaluarea chiar a rulat: un test care nu se execută din cauza unei dependențe lipsă trece la fel de tăcut ca unul care se execută cu succes, iar diferența dintre cele două situații este exact diferența dintre a ști și a crede că știi.

Schimbare injectată	Rangul primului candidat care o acoperă
Structură demolată	10
Obiect nou apărut	1
Vegetație îndepărtată	9
Șanț de excavație	2

Tabelul 3. Rezultatul curent: 4 din 4 schimbări regăsite, adâncime maximă rangul 10

Toate cele patru schimbări sunt regăsite în primii zece candidați, iar toți cei optsprezece candidați propuși peste zonele reale cad efectiv pe o schimbare. Cifra nu trebuie însă supraevaluată: este obținută pe o pereche sintetică, în care modificările au fost introduse de mine, într-o singură imagine. Ea demonstrează că metoda funcționează pe un caz controlat, nu că ar funcționa la fel pe zboruri reale — vezi capitolul 11.

6. Validarea datelor de intrare

Un zbor prost executat nu poate fi salvat prin prelucrare. Dacă fotografiile sunt neclare, dacă lipsesc coordonatele din metadate sau dacă suprapunerea dintre cadre este insuficientă, reconstrucția eșuează — dar eșuează după ore de calcul. Componenta de validare există pentru ca acest verdict să vină în câteva secunde, înainte de prelucrare, și însoțit de motivul concret.

6.1. Verificările efectuate

- Claritatea fiecărei fotografii, estimată prin varianța laplacianului pe imaginea convertită în tonuri de gri și redimensionată. O imagine neclară are puține treceri bruște de intensitate, deci o varianță mică a laplacianului.
- Prezența și consistența coordonatelor GPS din metadatele EXIF. O fotografie fără poziție nu poate fi așezată în bloc și nu poate participa la estimarea suprapunerii.
- Suprapunerea dintre cadre consecutive, calculată din distanța dintre pozițiile GPS și din amprenta la sol a fiecărui cadru, care rezultă la rândul ei din altitudinea de zbor, distanța focală și lățimea senzorului.

Fișierele care nu pot fi citite sunt marcate ca atare și raportate, nu propagă o excepție care ar opri validarea întregului set. Toate pragurile sunt parametri cu valori implicite declarate într-un singur loc, nu constante ascunse în mijlocul logicii, tocmai pentru că valorile corecte se vor cunoaște abia când vor exista zboruri reale.

6.2. O eroare de model, descoperită prin citire atentă

Calculul suprapunerii folosea altitudinea din metadatele EXIF ca înălțime de zbor deasupra terenului. Altitudinea scrisă în EXIF este însă, în cazul obișnuit, înălțimea deasupra nivelului mării. Diferența nu este academică: la un teren aflat la 500 de metri altitudine și un zbor la 100 de metri deasupra lui, amprenta la sol calculată era de șase ori mai mare decât cea reală, iar suprapunerea estimată apărea ca 0,61 în locul valorii reale de 0,26.

Ceea ce face eroarea gravă este direcția ei. Greșeala mergea întotdeauna în sensul permisiv: un set de fotografii care ar fi trebuit respins primea verdict de acceptare. Un instrument de validare care greșește în direcția permisivă este mai rău decât absența lui, pentru că înlocuiește o incertitudine recunoscută cu o falsă certitudine. Componenta acceptă acum înălțimea deasupra terenului ca parametru explicit, iar comportamentul este documentat.

Această verificare nu a rulat niciodată pe fotografii reale de dronă: metadatele EXIF folosite în teste sunt scrise de mine, prin fixturi sintetice. Limitarea este consemnată explicit atât în registrul de decizii, cât și în capitolul 11 al acestei lucrări.

7. Accesibilitatea interfeței

Interfața a fost construită de la început pentru a fi utilizabilă și fără mouse, și fără vedere. Motivul nu este exclusiv etic: dacă beneficiarul final este o instituție publică sau un proiect cu finanțare publică, standardul european EN 301 549 devine o cerință legală, iar respectarea lui din construcție costă incomparabil mai puțin decât adăugarea ulterioară.

7.1. Măsurile implementate

- Navigare completă de la tastatură, cu ordine de parcurgere care urmează structura vizuală a paginii și cu indicator de focus vizibil pe fiecare element interactiv.
- Bara de secțiuni implementată ca listă de file cu tabindex mobil: o singură oprire de Tab pentru întreaga bară, iar selecția se schimbă cu săgețile. Anterior, parcurgerea barei consuma aproximativ cincisprezece apăsări de Tab înainte de a ajunge la conținut.
- Dialogul cu lista completă de anomalii folosește elementul nativ de dialog, mută focusul pe titlul dialogului la deschidere, dezactivează conținutul din spate și readuce focusul pe butonul de origine la închidere.
- Dimensiunea zonelor de acționare respectă cerința de mărime minimă a țintei, iar rapoartele de contrast au fost măsurate, nu estimate vizual.
- Reașezarea conținutului la o lățime echivalentă de 320 de pixeli, fără derulare pe orizontală, condiție verificată automat.

7.2. Problema centrală: harta

Cea mai serioasă problemă de accesibilitate a acestui proiect nu este un buton fără etichetă, ci faptul că rezultatul central al aplicației — poziția geografică a fiecărei anomalii — era disponibil exclusiv vizual. Harta este o suprafață de desen WebGL: pentru un cititor de ecran, ea pur și simplu nu există. Lista de candidați afișa rangul și scorul, niciodată locația. Consecința practică era că o persoană nevăzătoare putea parcurge întregul flux de încărcare și validare, dar nu putea consuma niciun rezultat de detecție.

Problema a fost tratată prin adăugarea unui echivalent textual al hărții: un tabel care conține, pentru fiecare anomalie, rangul, scorul, zona geografică și coordonatele, precum și prin anunțarea explicită a acțiunilor de selecție, astfel încât deplasarea hărții să fie însoțită de o descriere a ceea ce s-a selectat. Prezența acestui echivalent este verificată automat.

7.3. Ce rămâne neverificat

Testele automate de accesibilitate acoperă în jur de o treime din criteriile standardului: prind o etichetă lipsă, un contrast sub prag, un identificator duplicat. Nu pot spune dacă ordinea de

citire are sens, dacă un anunț se înțelege sau dacă cineva poate duce efectiv o sarcină la capăt. Aplicația nu a fost niciodată parcursă cu un cititor de ecran real. Scenariul de verificare este redactat, cu douăzeci și șapte de pași grupați pe șase teme și cu criteriu explicit de eșec pentru fiecare, dar parcurgerea lui rămâne pasul următor.

8. Testare și integrare continuă

Proiectul conține 90 de teste executate cu pytest, pentru partea de prelucrare și pentru interfața de programare, și 48 de teste de browser executate cu Playwright, pentru interfața de utilizator. Toate se execută automat la fiecare modificare trimisă în depozit, împreună cu remăsurarea recall-ului pe perechea cu adevăr cunoscut.

8.1. Două lecții despre teste

Prima: un test poate trece fără să demonstreze nimic. Testul care verifică algoritmul central al aplicației compară imaginea de referință cu ea însăși. Trecea impecabil și nu putea eșua niciodată, indiferent cât de greșită ar fi fost implementarea. A fost rescris cu o schimbare reală injectată și cu verificarea rangului acesteia.

A doua: un test care nu se execută trece la fel de tăcut ca unul care reușește. Măsurarea recall-ului depinde de biblioteci care pot lipsi din mediul de integrare; în acest caz testul ar fi fost sărit, iar rezultatul general ar fi rămas verde. Verificarea din integrarea continuă inspectează acum textul emis de rulare și eșuează dacă cifra de recall nu apare — un test care nu s-a executat este tratat ca eșec, nu ca succes.

8.2. Contractul dintre componente

Testele de interfață rulează pe un serviciu simulat, ca să fie rapide și independente de rețea. Riscul evident este ca serviciul simulat să se depărteze în timp de cel real, iar testele să continue să treacă pe o realitate care nu mai există. Un test separat compară structura răspunsurilor celor două și eșuează la prima divergență.

9. Probleme întâmpinate și modul de rezolvare

9.1. Memorie epuizată la punerea în funcțiune

Prelucrarea funcționa local, dar pe planul gratuit de găzduire, limitat la 512 MB de memorie, procesul era oprit de sistem cu cod de ieșire 137. Cauza era încărcarea integrală a ambelor rastere în memorie. Rezolvarea a avut două etape: citirea rasterului pe ferestre, în locul citirii complete, și reducerea imaginii la 3000 de pixeli pe latura lungă înainte de prelucrare.

Detaliul util din această problemă nu este soluția, ci felul în care s-a confirmat direcția: după prima etapă, procesul murea în continuare, dar la 2 minute și 28 de secunde în loc de 16 secunde. Momentul mutat al eșecului era dovada că schimbarea acționase asupra cauzei corecte, chiar dacă simptomul nu dispăruse încă. Efectul secundar al reducerii de rezoluție a fost neașteptat de favorabil: zona de vegetație, până atunci nedetectată, a devenit detectabilă, iar recall-ul a urcat de la trei la patru din patru.

9.2. Un panou care acoperea harta

Dialogul cu lista completă de anomalii rămânea desenat peste hartă și după închidere, acoperind o suprafață de 1440×675 pixeli. Aplicația părea complet nefuncțională. Testul existent verifica faptul că dialogul era marcat ca închis în starea internă — și această verificare trecea — dar nu verifica niciodată dacă elementul mai era vizibil pe ecran. Lecția reținută este că un test trebuie să verifice ceea ce vede utilizatorul, nu ceea ce crede programul despre sine.

9.3. Georeferențiere care funcționa din întâmplare

Componenta de servire a dalelor trata coordonatele rasterului ca fiind direct comparabile cu cele ale sistemului folosit de harta web, fără nicio transformare de proiecție. Funcționa impecabil — dar numai pentru că imaginea de test era, întâmplător, într-un sistem de coordonate compatibil. Orice ortofotoplan românesc livrat în proiecție Stereografic 1970 ar fi fost afișat în alt loc de pe glob. Cazul este reprezentativ pentru o categorie de defecte periculoase: cele care nu se manifestă pe datele de test disponibile.

9.4. Un mesaj de accesibilitate absurd

Echivalentul textual al hărții anunța distanțele față de centrul zonei folosind direct valorile din sistemul de coordonate, ceea ce producea formulări de tipul „la aproximativ 544820807340 de metri de centru”. Textul era prezent, deci verificările automate treceau; era însă complet lipsit de sens pentru un utilizator. Prezența unui text alternativ nu garantează utilitatea lui, iar aceasta este exact zona în care testarea automată se oprește.

9.5. O regulă de proces, învățată dintr-un incident

Pe 17 august, un agent de execuție folosit pentru sarcini repetitive de scriere de cod a comis și a trimis pe ramura principală aproximativ cinci sute de linii nerevizuite, din proprie inițiativă. Nimic nu s-a pierdut, dar istoricul depozitului a trebuit reconstituit commit cu commit. Regula introdusă imediat după și respectată până la final: instrumentele automate pot edita fișiere, dar operațiile care modifică istoricul comun rămân la om, după revizuire. Regula este scrisă în fișierul de instrucțiuni al proiectului, împreună cu motivul și cu data incidentului, ca să nu fie relaxată din uitare.

9.6. Ce au în comun aceste defecte

Cu o singură excepție, niciunul dintre defectele întâmpinate nu a fost o greșală de algoritm. Au fost verificări care păreau să confirme, dar nu confirmau nimic; suprapuneri între ipoteze implicite și date de test alese convenabil; instrumente care greșeau în direcția permisivă. Concluzia pe care o duc mai departe din acest stagiu este că întrebarea „ce anume ar trebui să eșueze dacă implementarea ar fi greșită?” este mai productivă decât întrebarea „trece testul?”.

10. Rezultate obținute

Indicator	Valoare
Schimbări regăsite pe perechea cu adevăr cunoscut	4 din 4, adâncime maximă rangul 10
Precizia candidaților peste zonele reale	18 din 18 (100%)
Linii de cod de aplicație	aproximativ 7.100
Linii de cod de test	aproximativ 3.000
Teste automate	90 pytest + 48 Playwright
Rute HTTP expuse	13
Commituri în depozit	63
Decizii tehnice consemnate	18
Forme de livrare	aplicație web publică și program Windows de sine stătător

Tabelul 4. Situația proiectului la încheierea stagiului

Dincolo de cifre, rezultatul concret al stagiului este că aplicația poate fi deschisă și folosită de altcineva decât autorul ei, fără instalare și fără explicații prelabile, iar afirmațiile despre calitatea detecției pot fi verificate independent, prin rularea unei singure comenzi.

11. Limitări și direcții de dezvoltare

11.1. Ce nu s-a realizat

- Nicio linie din această aplicație nu a atins vreodată date reale de zbor. Toate rezultatele sunt obținute pe un ortofotoplan public și pe o pereche construită în laborator.
- Faza de fotogrammetrie — reconstrucția ortofotoplanului pornind de la fotografiile brute — nu a fost implementată. Ea cere resurse de calcul care nu erau disponibile, iar motorul gratuit avut în vedere, OpenDroneMap, ridică o problemă de licențiere neclarificată.
- Detecția presupune că cele două imagini sunt deja aliniate. Pe zboruri reale, alinierea trebuie realizată explicit, altfel diferența de trăsături măsoară deplasarea, nu schimbarea.
- Aplicația nu are autentificare. Este o alegere corectă atât timp cât datele sunt sintetice și publice, dar trebuie schimbată înainte de prima imagine reală a unui amplasament.
- Verificarea cu un cititor de ecran real nu a fost efectuată.

11.2. Direcții imediate

Prima direcție este calibrarea pe date reale: pragul de sensibilitate și dimensiunea peticului sunt deja parametri, iar valorile lor corecte depind de intervalul dintre zboruri. La o săptămână distanță, diferențele de iluminare și umbră domină semnalul util; la un sezon distanță, creșterea naturală a vegetației îneacă schimbările care contează. A doua este alinierea automată a celor două rastere. A treia este reproiectarea corectă la servirea dalelor, necesară pentru orice ortofotoplan livrat în sistemul de coordonate național.

Toate aceste direcții sunt consemnate ca sarcini deschise în documentația proiectului, fiecare cu motivul pentru care nu a fost tratată și cu ceea ce ar debloca-o.

12. Concluzii

Stagiul a produs o aplicație funcțională, publică și verificabilă, care rezolvă o problemă reală: reducerea comparării manuale a două ortofotoplanuri la verificarea unei liste scurte de zone candidate. Metoda aleasă — detecția de anomalii pe trăsături de diferență — regăsește toate schimbările cunoscute în primii zece candidați propuși, iar cifra este remăsurată automat la fiecare modificare a codului.

Din punct de vedere profesional, partea cea mai instructivă a stagiului nu a fost scrierea codului, ci disciplina din jurul lui. Un rezultat notat greșit și corectat public după verificarea pe altă mașină, un test care trecea fără să demonstreze nimic, o componentă de validare care greșea în direcția permisivă, o funcționalitate care mergea din întâmplare pentru că datele de test erau alese convenabil — fiecare dintre acestea a fost consemnată cu simptomul, cauza și reparația ei. Documentația scrisă pe parcurs, nu la final, a fost instrumentul care a făcut posibilă reluarea lucrului pe altă mașină, în altă zi, fără pierdere de context.

La fel de importantă este lista limitărilor. Aplicația nu a văzut niciodată date reale de zbor, nu realizează fotogrammetrie, presupune imagini aliniate și nu a fost parcursă cu un cititor de ecran. Enumerarea explicită a acestor limite nu diminuează lucrarea, ci o face utilizabilă: cine o preia știe exact de unde continuă și ce nu are voie să presupună.

Bibliografie

- Liu, F. T., Ting, K. M., Zhou, Z.-H., „Isolation Forest”, Proceedings of the 8th IEEE International Conference on Data Mining, 2008.
- Pedregosa, F. et al., „Scikit-learn: Machine Learning in Python”, Journal of Machine Learning Research, vol. 12, 2011.

- Open Geospatial Consortium, „Cloud Optimized GeoTIFF (COG) Standard”, 2023.
- GDAL/OGR contributors, GDAL — Geospatial Data Abstraction Library, documentație oficială, gdal.org.
- Rasterio, documentație oficială, rasterio.readthedocs.io.
- FastAPI, documentație oficială, fastapi.tiangolo.com.
- MapLibre GL JS, documentație oficială, maplibre.org.
- W3C, „Web Content Accessibility Guidelines (WCAG) 2.2”, Recomandare W3C, 2023.
- ETSI, EN 301 549 — Cerințe de accesibilitate pentru produse și servicii TIC, v3.2.1.
- OpenDroneMap, documentație oficială, docs.opendronemap.org.
- Documentația internă a proiectului (Obsidian): plan de implementare, registru de decizii, jurnal de lucru, listă de probleme și rezolvări, scenariu de verificare cu cititor de ecran.

Anexă. Instantanee din perioada practicii

Conform modelului, dosarul de practică este însoțit de câteva instantanee din perioada practicii. Stagiul s-a desfășurat pe partea de dezvoltare software, așa că imaginile de mai jos sunt capturi din aplicația realizată și din documentația ținută pe parcursul lucrului.

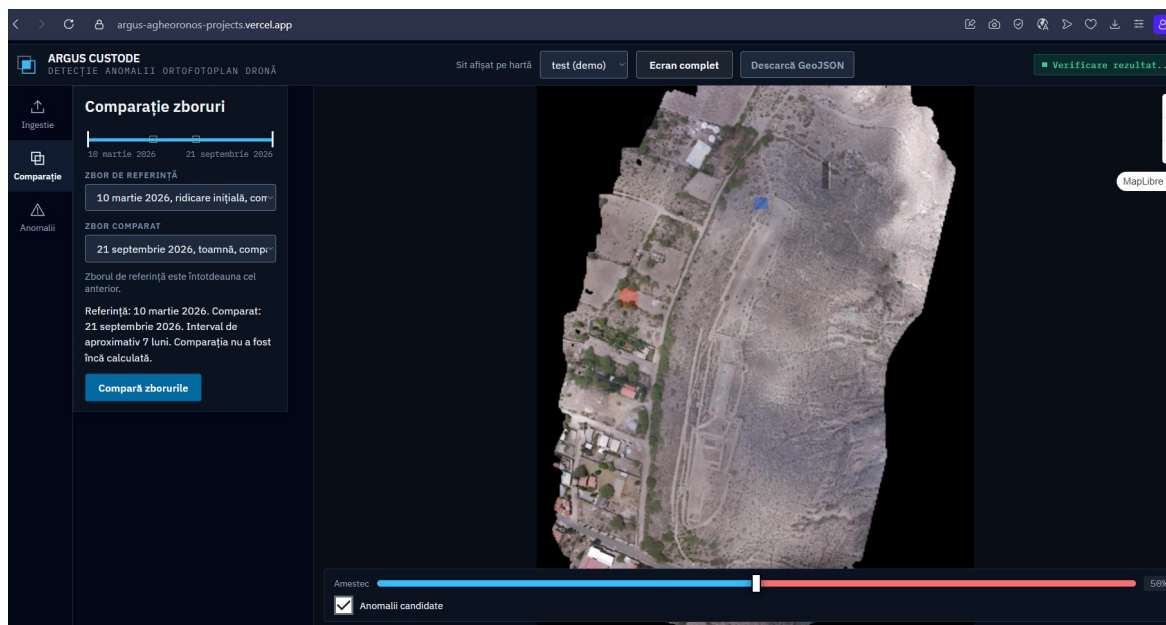


Figura 1. Aplicația în funcțiune la adresa publică, fila Comparație.

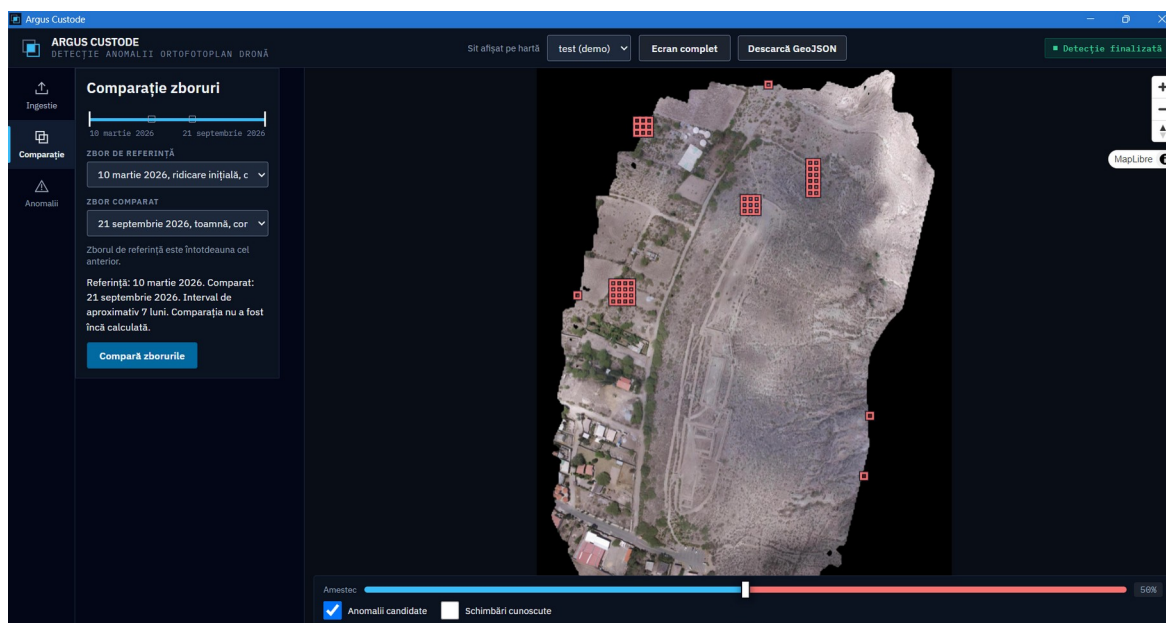


Figura 2. Aplicația de desktop pentru Windows, rulând ca fereastră proprie.

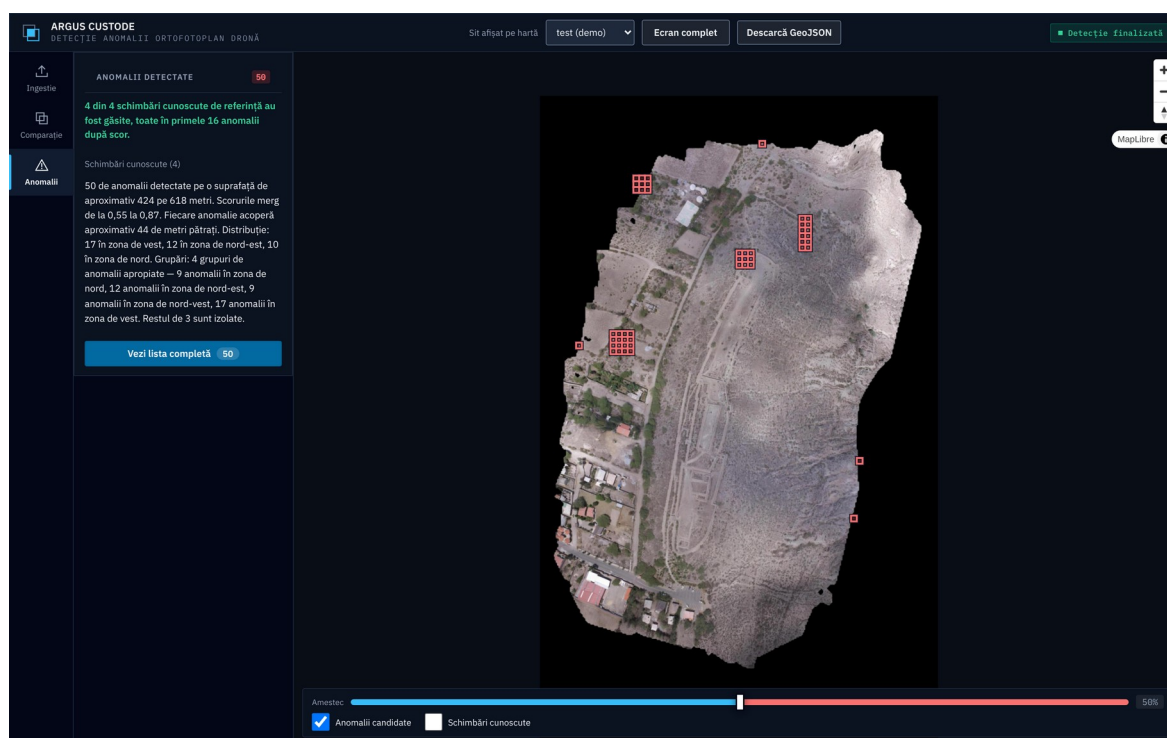


Figura 3. Harta cu anomaliile suprapuse, colorate după scor, și panoul de candidați ordonat după rang.

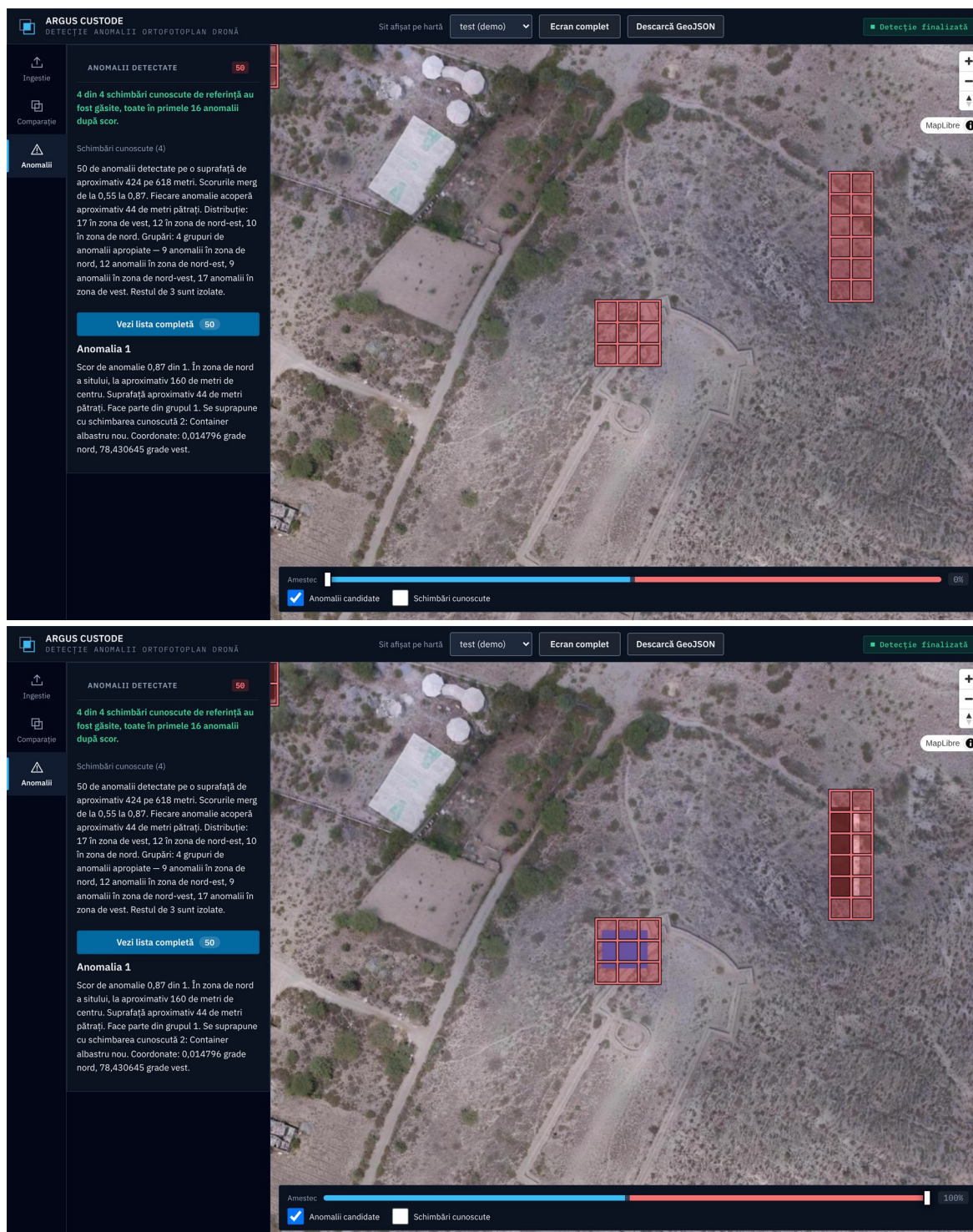
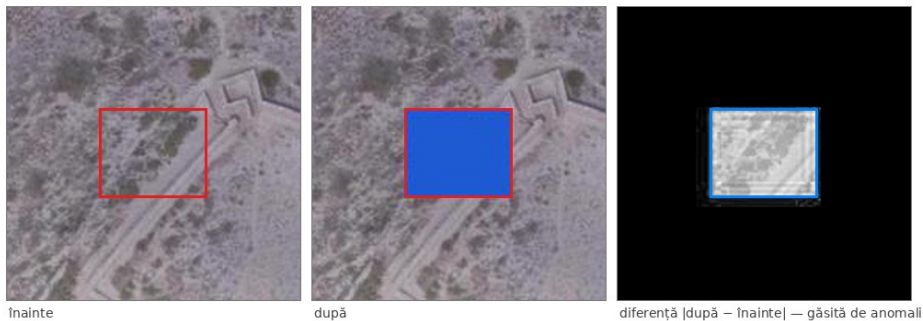


Figura 4. Aceeași zonă înainte și după: containerul albastru apare numai în ortofotoplanul „după”, în interiorul poligonului de anomalie.

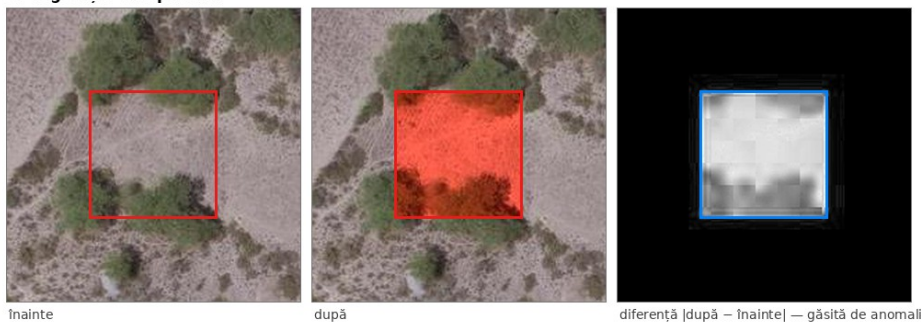
1. Structură demolată



2. Container albastru nou



3. Vegetație îndepărtată



4. Șanț de excavație

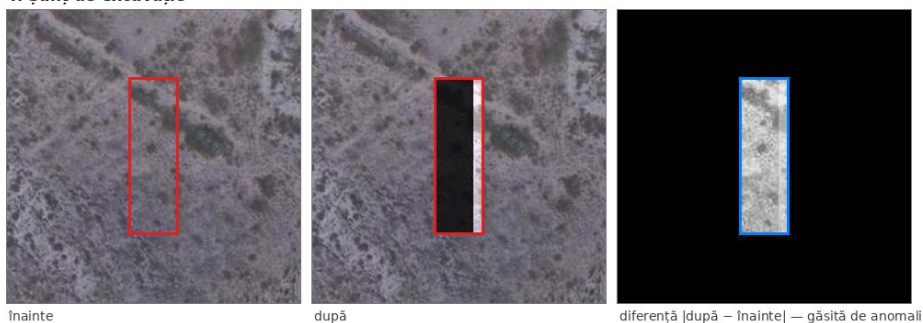


Figura 5. Cele patru schimbări injectate. Pentru fiecare: decupajul înainte, decupajul după și amplitudinea diferenței, cu poligonul de adevăr desenat peste.


```
$ python -m pytest tests/ -q

.....
recall benchmark (seed 20260826, top_n=50)
  structura demolata rang 10
  obiect nou aparut rang 1
  vegetatie indepartata rang 9
  sant de excavatie rang 2
  recall 4/4, adancime maxima 10
... precizie 18/18 candidati pe o schimbare reala (100%)
.....S..... [ 82%]
..... [100%]
86 passed, 1 skipped in 7.98s

$ npx playwright test --list

Total: 48 tests in 6 files
```

teste pe fișier

test_timeline.py	17
test_ingest.py	14
test_security.py	13
test_regression_audit.py	10
test_api_contract.py	8
test_ingest_api.py	8
test_recall_benchmark.py	4
test_desktop.py	4
test_desktop_bundle.py	4
test_features.py	3
test_tiles.py	3
test_main.py	2
test_detect.py	1

test_desktop_bundle.py (4 teste)
pornește executabilul Windows și
rulează numai pe Windows.
Total colectat: 91 de teste.

Figura 6. Rularea suitei de teste: 86 de teste pytest trecute și 48 de teste Playwright.

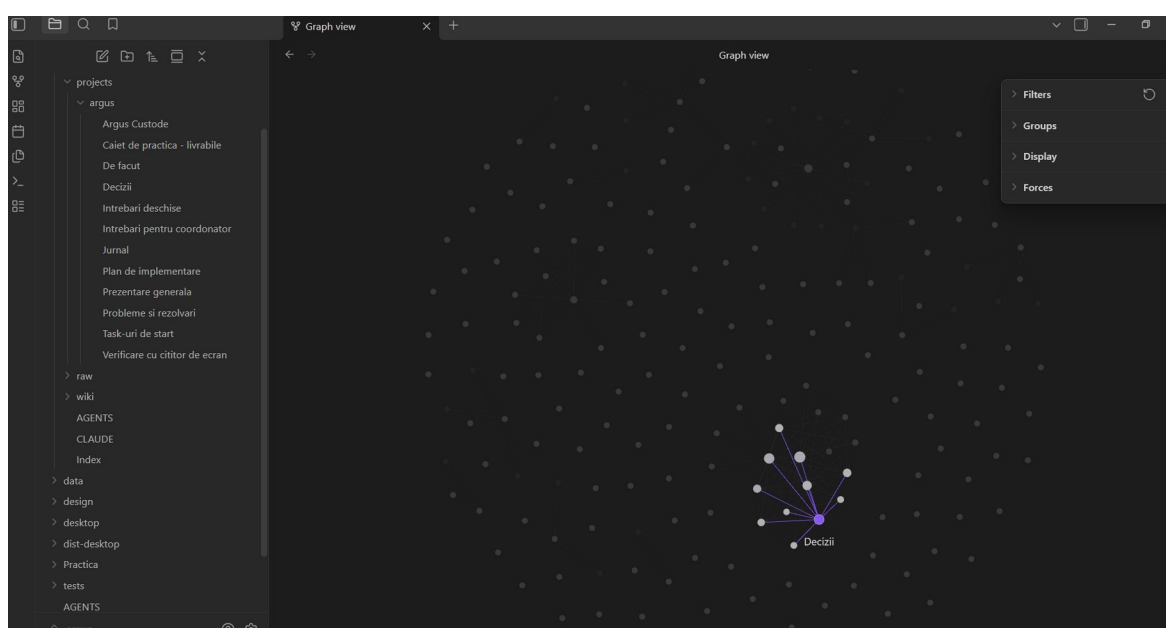


Figura 7. Registrul de decizii din documentația proiectului și graful legăturilor dintre note.